

**INSTITUTO
FEDERAL**
Santa Catarina

Introdução a Paradigmas de Programação

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PROGRAMAÇÃO
FRONTEND II

Análise e Desenvolvimento
de Sistemas

cores

#111027

#277756

#16ABCD

#FFF4EC

#C74E23

fontes

Fira Sans Extra Condensed

Ubuntu

Roboto Mono



Paradigmas de Programação

- estilos ou abordagens de programação
- conjuntos de conceitos, técnicas e princípios
- guiam o desenvolvimento de software
- definem como os problemas são resolvidos

Programação Imperativa

visão geral

- paradigma mais antigo
- semelhança ao padrão imperativo das linguagens naturais
- foco em **como** o computador deve executar as instruções
- comandos executados na ordem em que aparecem na memória
- Turing completa



Programação Imperativa

características

- estado do sistema
- instruções sequenciais
- controle de fluxo
- variáveis e atribuições
- efeitos colaterais
- abstração procedural
- mutabilidade
- entrada e saída (i/o)

exemplos

```
#include <stdio.h>

void dobrarElementos(int arr[], int tamanho) {
    for (int i = 0; i < tamanho; i++) {
        arr[i] *= 2;
    }
}

int main() {
    int numeros[] = {1, 2, 3, 4};
    int tamanho = sizeof(numeros)/sizeof(numeros[0]);
    dobrarElementos(numeros, tamanho);
    for (int i = 0; i < tamanho; i++) {
        printf("%d ", numeros[i]);
    }
    return 0;
}
```



Programação Imperativa

características

- estado do sistema
- instruções sequenciais
- controle de fluxo
- variáveis e atribuições
- efeitos colaterais
- abstração procedural
- mutabilidade
- entrada e saída (i/o)

exemplos

```
let contador = 0;

function incrementarContador() {
  contador++;
  console.log(`O contador agora é: ${contador}`);
}

incrementarContador();
incrementarContador();
```



Programação Declarativa

visão geral

- foco em **o que** deve ser feito, e não como deve ser feito
- sintaxe intuitiva
- permite a definição de restrições e regras
- aplicações
 - inteligência artificial
 - acesso de informações em BD



VHDL



Programação Declarativa

características

- declaração de intenção
- imutabilidade
- funções puras
- sem controle de fluxo explícito
- recursão
- abstração de estado
- expressões em vez de instruções
- idempotência

exemplos

```
SELECT nome, idade FROM clientes WHERE idade > 30;
```

```
SELECT * FROM produtos WHERE estoque > 0;
```

```
SELECT categoria, COUNT(*) AS total_produtos  
FROM produtos  
GROUP BY categoria;
```

```
SELECT nome, salario  
FROM empregados  
WHERE salario > 3000  
ORDER BY salario DESC;
```

```
SELECT nome FROM clientes WHERE cidade = 'São Paulo';
```



Programação Declarativa

características

- declaração de intenção
- imutabilidade
- funções puras
- sem controle de fluxo explícito
- recursão
- abstração de estado
- expressões em vez de instruções
- idempotência

exemplos

```
const numeros = [1, 2, 3, 4, 5];  
const dobrados = numeros.map(num => num * 2);  
console.log(dobrados); // [2, 4, 6, 8, 10]
```

```
const resultado = condicao ? 'Verdadeiro' : 'Falso';
```

```
let numeros = [1, 2, 3, 4, 5, 6];  
let pares = numeros.filter(num => num % 2 === 0);  
let soma = pares.reduce((acc, num) => acc + num, 0);
```

```
console.log(pares); // [2, 4, 6]  
console.log(soma); // 12
```



Programação Funcional

visão geral

- foco em **o que** deve ser feito, e não como deve ser feito
- avaliação de funções matemáticas
- evita o uso de estado e de mutabilidade
- a maioria das linguagens modernas incorporam aspectos funcionais



Programação Funcional

características

- funções como cidadãos de 1ª classe
- imutabilidade
- funções puras
- transparência referencial
- recursão
- composição de funções
- avaliação preguiçosa
- uso de coleções imutáveis

exemplos

```
val somar = (a: Int, b: Int) => a + b
def aplicarOperacao(operacao: (Int, Int) => Int, x: Int,
y: Int): Int = operacao(x, y)
println(aplicarOperacao(somar, 3, 4))
```

```
val lista = List(1, 2, 3)
val novaLista = lista :+ 4
println(lista) // List(1, 2, 3)
println(novaLista) // List(1, 2, 3, 4)
```

```
def multiplicar(a: Int, b: Int): Int = a * b
println(multiplicar(3, 4)) // 12
```

```
val dobro = (x: Int) => x * 2
println(dobro(3) + dobro(3)) // 12
```



Programação Funcional

características

- funções como cidadãos de 1ª classe
- imutabilidade
- funções puras
- transparência referencial
- recursão
- composição de funções
- avaliação preguiçosa
- uso de coleções imutáveis

exemplos

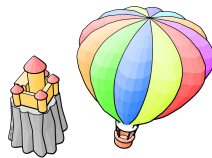
```
const multiplica = (a, b) => () => a * b;  
const resultado = multiplica(3, 4);  
// Não executa a multiplicação ainda  
console.log(resultado());  
// 12, a multiplicação é executada aqui  
  
const fat = n => (n === 0 ? 1 : n * fat(n - 1));  
console.log(fatorial(5)); // 120  
  
const capitalizar = texto => texto.toUpperCase();  
const exclamar = texto => `${texto}!`;  
const saudar = texto => exclamar(capitalizar(texto));  
  
console.log(saudar("olá")); // "OLÁ!"
```



Programação Orientada a Objetos

visão geral

- abordagem centrada em objetos para modelar o mundo real
- objetos como instâncias de classes
- objetos possuem atributos (propriedades) e comportamento (métodos)
- código mais modular, reutilizável e fácil de entender



Programação Orientada a Objetos

características

- objetos e classes
- encapsulamento
- herança
- polimorfismo
- abstração
- reusabilidade
- composição

exemplos

```
class Carro {  
    String modelo;  
    int ano;  
  
    void buzinar() {  
        System.out.println("Bip bip!");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Carro meuCarro = new Carro();  
        meuCarro.modelo = "Fusca";  
        meuCarro.ano = 1968;  
        meuCarro.buzinar();  
    }  
}
```



Programação Orientada a Objetos

características

- objetos e classes
- encapsulamento
- herança
- polimorfismo
- abstração
- reusabilidade
- composição

exemplos

```
class Carro {  
  constructor(modelo, ano) {  
    this.modelo = modelo;  
    this.ano = ano;  
  }  
  
  buzinar() {  
    console.log("Bip bip!");  
  }  
}  
  
const meuCarro = new Carro("Fusca", 1968);  
console.log(meuCarro.modelo);  
meuCarro.buzinar();
```

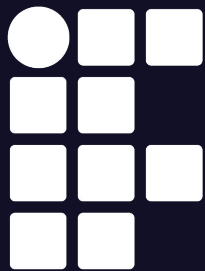


Paradigmas de Programação em JavaScript

linguagem multiparadigma

- imperativa
 - tarefas simples ou de controle de fluxo
- funcional
 - manipulação de dados e cálculos matemáticos
- orientada a objetos
 - modelagem de entidades do mundo real e interfaces de usuário complexas





**INSTITUTO
FEDERAL**
Santa Catarina

Introdução a Paradigmas de Programação

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PROGRAMAÇÃO
FRONTEND II

Análise e Desenvolvimento
de Sistemas